
Temporal Regularized Learning: A Neuron-Local Rule for Deep Slow-Feature Learning

Anonymous Authors

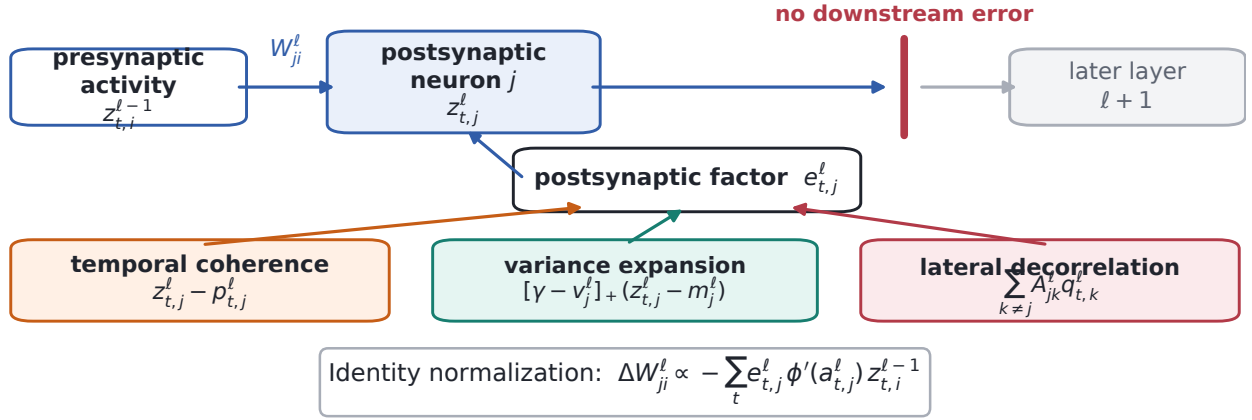
Abstract

Slow features can reveal latent factors that persist while individual observations change. Minimizing temporal variation alone, however, also makes a constant representation optimal. Slow Feature Analysis (SFA) excludes this solution by preserving variance and decorrelating feature coordinates. We ask whether the same principle can train a deep nonlinear encoder without propagating errors between layers. Temporal Regularized Learning (TEREL) realizes SFA through three soft signals at each neuron: temporal coherence, gated variance expansion, and lateral decorrelation. Canonical TEREL retains gradients between adjacent activations within a short chunk; TEREL-S detaches the preceding activation and therefore needs only fixed temporal state.

Under constructed, label-ordered MNIST supervision, canonical TEREL reaches $97.30 \pm 0.07\%$ test accuracy, compared with $98.34 \pm 0.08\%$ for data-presentation-matched backpropagation, $96.98 \pm 0.10\%$ for a matched Local SupCon learner, and $95.35 \pm 0.19\%$ for a normalization-matched random encoder. Mechanism ablations produce the collapse modes predicted by the three terms, and the lagged lateral direction has mean cosine alignment 0.76 with its same-batch target. TEREL-S reaches $96.29 \pm 0.07\%$ test accuracy; a separate batch-size-one configuration reaches $95.14 \pm 0.04\%$ validation accuracy without retaining a temporal graph. A chronological PAMAP2 test is inconclusive. The evidence therefore supports neuron-local learning in the controlled MNIST regime, not a general claim about natural temporal order.

A Information available to one feedforward weight update

Signals may arrive from the same layer, but no error arrives from a later layer.



B The temporal distinction between the variants

	temporal reference	gradient through time	execution consequence
canonical TeReL	$p_t = z_{t-1}$ in the chunk	retained across adjacent samples	short chunk graph must be retained
TeReL-S	$p_t = \text{sg}(z_{t-1})$	stopped at the stored reference	fixed temporal state; no time graph

Figure 1: Why TeReL is neuron-local. **A:** A feedforward weight uses its presynaptic activity and a postsynaptic factor formed from three signals; a same-layer lateral signal is allowed, but no error arrives from a later layer. The factorization shown is exact with identity normalization. **B:** Canonical TeReL retains gradients through adjacent samples, whereas TeReL-S detaches the preceding activation. BatchNorm is omitted because it is a configuration choice, not a fourth learning signal; [Section 2.3](#) specifies its within-batch dependency.

1 Introduction

Observations often change more rapidly than the causes that generate them. Object identity can persist across changes in viewpoint and illumination; an activity can persist across noisy wearable-sensor measurements; and a dynamical state can evolve more slowly than its observation noise. Temporal proximity can therefore identify factors that remain stable without requiring two manually designed views of each observation. Yet temporal coherence alone cannot define a useful representation: the constant map is perfectly slow.

Slow Feature Analysis (SFA) removes this degeneracy by minimizing temporal derivatives subject to zero-mean, unit-variance, and decorrelation constraints [Wiskott and Sejnowski \[2002\]](#). Regularized self-supervised methods use a closely related decomposition. They couple an invariance term to variance and covariance penalties that prevent collapse [Bardes et al. \[2022\]](#), [Zbontar et al. \[2021\]](#). These principles are established. The unresolved question studied here is how to realize them in a deep nonlinear encoder whose parameters receive no error from later layers. A second question is what additional restriction makes the resulting rule executable with bounded temporal history.

Temporal Regularized Learning (TeReL) assigns each layer its own soft-SFA objective. A temporal term attracts the current activation to the preceding activation. A gated variance term expands

a coordinate while its detached running variance remains below a target. A dense lateral operator tracks off-diagonal second moments and supplies a lagged approximation to the same-batch covariance gradient. Stop-gradient boundaries between layers confine credit assignment to the layer that produced each activation.

We call the rule neuron-local in space because the update of a feedforward weight may use its presynaptic activity, quantities at its postsynaptic neuron, and signals delivered through same-layer lateral connections, but no error from a later layer. Neuron-local does not mean neuron-independent: the dense lateral operator couples all units within a layer and requires $O(D^2)$ state at width D .

Two variants separate this spatial locality from locality across time. Canonical TEREL retains the preceding activation in a short within-chunk graph, so its update is neuron-local but not temporally online. TEREL-S detaches the preceding activation and retains only fixed-size state. Either rule may be evaluated on a minibatch; batching does not define the spatial locality claim.

The source of temporal order also determines the scope of the evidence. Our main experiment arranges MNIST training examples into label-homogeneous chunks. Labels do not appear in the TEREL objective, but they do construct its temporal supervision. This experiment therefore tests the learning rule under a controlled, task-aligned temporal relation; it is not unlabeled self-supervision. We also test chronological wearable-sensor streams while withholding activity classes from the encoder loss. That secondary experiment is inconclusive, so we do not infer that arbitrary natural adjacency reveals task structure.

The paper makes the following contributions:

1. We formulate TEREL as a deep neuron-local soft-SFA rule and specify its relation to SFA, Incremental SFA, Bio-SFA, VICReg, and gradient-isolated predictive, contrastive, and blockwise learning.
2. We derive the exact activation and parameter gradients with and without temporal detachment, including the Jacobian of the within-layer normalizer. We specify the detached state updates, decompose the error of the lagged lateral proxy, and measure its alignment with the same-batch direction.
3. Under constructed MNIST temporal supervision, canonical TEREL attains 97.30% accuracy, compared with 98.34% for backpropagation with the same number of data presentations and 95.35% for a normalization-matched random encoder. Local SupCon and direct-covariance controls delimit what can be attributed to the soft-SFA construction.
4. We train TEREL-S one sample at a time with fixed state and no retained temporal graph. We report parameters, optimizer state, dynamic state, peak memory, operation proxies, and throughput as separate quantities.
5. We separate validation-based selection from test evaluation and report matched architectures and readouts, five evaluation seeds, raw results, uncertainty intervals, and artifact-level provenance.

2 Temporal Regularized Learning

2.1 From constrained SFA to a soft local objective

Let x_t denote a temporal stream. On a local minibatch $\mathcal{B}$, layer ℓ computes

$$a_t^\ell = W^\ell z_t^{\ell-1} + c^\ell, \quad h_{\mathcal{B}}^\ell = N_{\eta^\ell, \mathcal{B}}(a_{\mathcal{B}}^\ell), \quad z_t^\ell = \phi(h_t^\ell). \quad (1)$$

Normalization is not part of the TEREL objective. The map N may be the identity or a within-layer normalizer with affine parameters η^ℓ . We use training-mode BatchNorm in the canonical MNIST configuration because it was selected empirically as an optimization choice; TEREL-S instead uses the identity or a bounded streaming normalizer. BatchNorm operates separately at each neuron coordinate but couples the examples in $\mathcal{B}$ at that coordinate. It therefore changes the within-batch dependency, not the path of credit across neurons or depth.

SFA seeks features that vary slowly without becoming constant or redundant. A standard formulation minimizes $\mathbb{E}\|z_t - z_{t-1}\|_2^2$ subject to zero mean, unit variance, and decorrelated outputs [Wiskott and Sejnowski \[2002\]](#). TEREL retains these three functions but replaces the hard population constraints by soft terms computed from detached running state. The SFA criterion is prior work; the contribution is its realization as a deep neuron-local learning rule.

Consider B activations $z_t \in \mathbb{R}^D$ from one layer. A Boolean flag b_t marks the first sample of a subject, recording, or constructed chunk. We define $I_t = 1 - b_t$ and also set $I_t = 0$ when no stored predecessor is available. Let $N_I = \sum_t I_t$. Before minibatch k , the layer holds the detached state $(m_{k-1}, v_{k-1}, A_{k-1})$. Suppressing the minibatch index, the local objective is

$$L_S = \frac{1}{N_I D} \sum_{t: I_t=1} \|z_t - p_t\|_2^2, \quad (2)$$

$$L_V = -\frac{1}{BD} \sum_{t=1}^B \sum_{j=1}^D [\gamma - v_j]_+ (z_{tj} - m_j)^2, \quad (3)$$

$$L_C = \frac{1}{BD} \sum_{t=1}^B (z_t - m)^\top A q_t, \quad (4)$$

$$L = \lambda_S L_S + \lambda_V L_V + \lambda_C L_C. \quad (5)$$

When $N_I = 0$, both L_S and its gradient are zero. Canonical TEREL sets $p_t = z_{t-1}$ within a short chunk and $q_t = \text{sg}(z_t - m)$. The input to the next layer is detached, so the temporal graph cannot transmit an error across depth. TEREL-S instead sets $p_t = \text{sg}(z_{t-1})$, or uses the detached final activation from the preceding call. In both variants, a boundary flag removes the corresponding temporal pair.

The negative sign in [Eq. \(3\)](#) determines the action of the variance term. Treating $g_j = [\gamma - v_j]_+$ as fixed state, minimizing L_V moves an active coordinate away from its running mean until its variance reaches γ . Thus L_V is a local gradient surrogate for the SFA variance constraint, not the usual nonnegative scalar variance penalty of VICReg. Its behavior depends on the timescale of the running statistics, the normalizer, and the activation. We select these algorithmic hyperparameters on validation data.

2.2 Lateral decorrelation as a gradient proxy

For minibatch k , the detached off-diagonal second-moment operator is

$$C_k = \text{offdiag} \left(\frac{1}{B} \sum_t (z_{k,t} - m_{k-1})(z_{k,t} - m_{k-1})^\top \right). \quad (6)$$

Our direct-covariance control minimizes $L_{\text{direct}} = \|C_k\|_F^2 / D$. Because m_{k-1} is detached, the gradient with respect to $z_{k,t}$ is $4C_k(z_{k,t} - m_{k-1}) / (BD)$. TEREL cannot use C_k before the current activations have been observed. It therefore evaluates [Eq. \(4\)](#) with the stored operator A_{k-1} . Detaching A_{k-1} and $q_{k,t}$ gives the activation direction $A_{k-1}q_{k,t} / (BD)$. When $A_{k-1} \approx C_k$ and $q_{k,t} = z_{k,t} - m_{k-1}$, this direction agrees with the direct gradient up to the positive factor four. The relation is a lagged gradient approximation, not an equality of the two scalar objectives.

A shifted variant sets q_t to the preceding centered activation. Writing $u_t = z_t - m$, its deviation from the direct direction satisfies

$$\|Aq_t - Cu_t\|_2 \leq \|A - C\|_2 \|u_t\|_2 + \|A\|_2 \|q_t - u_t\|_2. \quad (7)$$

The first term measures error in the lateral operator. The second measures error from substituting the preceding activation for the current one. The bound gives no guarantee that either error is small, so the shifted rule is used only as an ablation.

2.3 Neuron-local gradient dependencies

Let $g = [\gamma - v]_+$. For TEREL-S, differentiating the three local terms with respect to the post-activation gives

$$e_t = \frac{2\lambda_S I_t}{N_I D} (z_t - p_t) - \frac{2\lambda_V}{BD} g \odot (z_t - m) + \frac{\lambda_C}{BD} Aq_t. \quad (8)$$

Set $r_s = e_s \odot \phi'(h_s^\ell)$, and denote the (s, t) block of the normalizer's minibatch Jacobian by $J_{st}^N = \partial h_s^\ell / \partial a_t^\ell$. Because the input $z_t^{\ell-1}$ to the learned layer is detached, the exact same-layer chain rule is

$$u_t = \sum_{s \in \mathcal{B}} (J_{st}^N)^\top r_s, \quad \frac{\partial L^\ell}{\partial W^\ell} = \sum_t u_t (z_t^{\ell-1})^\top, \quad \frac{\partial L^\ell}{\partial c^\ell} = \sum_t u_t, \quad \frac{\partial L^\ell}{\partial \eta^\ell} = \sum_s \left(\frac{\partial h_s^\ell}{\partial \eta^\ell} \right)^\top r_s. \quad (9)$$

For $N = \text{Id}$, $J_{st}^N = \delta_{st} I$, and the weight gradient reduces to the presynaptic outer product $\sum_t (e_t \odot \phi'(a_t^\ell)) (z_t^{\ell-1})^\top$. BatchNorm in training mode instead mixes the output-side factors across examples in $\mathcal{B}$ and also updates η^ℓ . For BatchNorm, this mixing remains within each postsynaptic coordinate; the lateral term supplies the only cross-neuron signal. Autograd computes the same-layer Jacobian exactly. Neither normalizer admits an error from a later layer, but only the identity normalizer admits the displayed per-example factorization.

Canonical TEREL keeps $p_t = z_{t-1}$ in the graph within a short chunk. The temporal term then contributes

$$\frac{2\lambda_S}{N_I D} [I_t (z_t - z_{t-1}) - I_{t+1} (z_{t+1} - z_t)], \quad (10)$$

where missing and boundary-crossing terms are omitted. The second term depends on z_{t+1} and is therefore not temporally online. Canonical TEREL is neuron-local in space; TEREL-S is neuron-local in space and local in time. Across minibatch calls, canonical TEREL also reads its stored predecessor as detached state. In the fixed MNIST construction, an average of 151.2 among 46,866 valid edges per data presentation cross a minibatch boundary. These edges constitute 0.32% of the total, or 9,073 edges over the 60-pass encoder schedule. Every other valid edge uses the live within-call gradient in Eq. (10).

2.4 State update and execution order

The loss for minibatch k is evaluated with the state available at the start of the step. After the optimizer update, the resulting activations update the state under `no_grad`:

$$m_k = \rho_m m_{k-1} + (1 - \rho_m) \bar{z}_k, \quad (11)$$

$$v_k = \rho_m v_{k-1} + (1 - \rho_m) \frac{1}{B} \sum_t (z_{k,t} - m_{k-1})^2, \quad (12)$$

$$A_k = \rho_A A_{k-1} + (1 - \rho_A) \text{offdiag} \left(\frac{1}{B} \sum_t (z_{k,t} - m_{k-1}) (z_{k,t} - m_{k-1})^\top \right). \quad (13)$$

The variance and lateral updates both use the pre-step mean m_{k-1} . When $\rho_A = 0$, Eq. (13) writes $A_k = C_k$ after the loss for minibatch k has already used A_{k-1} . The canonical lateral signal is therefore lagged by one minibatch. TEREL-S also stores the final activation and its centered value as detached temporal state. No activation graph survives the call.

Require: input batch/stream segment X , boundary flags b , layer parameters $\{W^\ell, c^\ell, \eta^\ell\}$, and states $\{m^\ell, v^\ell, A^\ell, p^\ell\}$

- 1: $h \leftarrow X$
- 2: **for** layer $\ell = 1, \dots, L$ **do**
- 3: $z^\ell \leftarrow \phi(N_{\eta^\ell, \mathcal{B}}(W^\ell \text{sg}(h) + c^\ell))$
- 4: construct detached predecessors p_i^ℓ ; remove pairs marked by b
- 5: evaluate the local objective L^ℓ in Eq. (5)
- 6: $h \leftarrow \text{sg}(z^\ell)$
- 7: **end for**
- 8: differentiate $\sum_\ell L^\ell$ within each layer, including $N_{\eta^\ell, \mathcal{B}}$, and update the parameters
- 9: **for** layer $\ell = 1, \dots, L$ **do**
- 10: update detached state by Eqs. (11) to (13)
- 11: **end for**

Algorithm 1: One optimization step of TEREL-S for a deep encoder.

2.5 Spatial versus temporal locality

Table 1: Method variants used in the experiments. Graph scope states the boundaries crossed by automatic differentiation.

Method	Temporal reference	Decorrelation	Graph scope
TEREL	live previous in chunk	lagged A_{k-1} , current q_t	same layer/chunk time
TEREL-S	detached previous	lagged A_{k-1} , current q_t	same layer/current step
Shifted proxy	detached previous	tracked A , previous q_t	same layer/current step
Direct control	live previous in chunk	exact in-batch $\ C_k\ _F^2$	same layer/chunk time
Local SupCon	label positives	supervised contrastive	same layer/current batch

For the identity normalizer, the update of W_{ji}^ℓ factors into the presynaptic activity $z_i^{\ell-1}$ and an output-side factor for neuron j . That factor uses neuron j 's temporal and variance state together with the signal delivered by row j of the lateral operator. The state updates are local to the same graph: m_j and v_j are coordinatewise, while each A_{jk} uses the centered activities at the endpoints of that lateral connection. The BatchNorm variant adds cross-example dependence at neuron j but no cross-depth error. We therefore describe the spatial rule as neuron-local while allowing dense same-layer communication. Canonical TEREL also propagates temporal credit within a short chunk; TEREL-S cuts this path and is online with bounded history at batch size one. The dependency graph alone implies neither biological plausibility nor hardware efficiency.

2.6 State and memory cost

At width D , the dynamic state of one TEREL layer contains a mean vector, a variance vector, a $D \times D$ lateral operator, the preceding activation, the preceding centered activation, and one validity flag.

The total is $D^2 + 4D + 1$ elements. A bounded streaming normalizer adds location and scale vectors containing $2D$ elements. This state is independent of stream length but quadratic in layer width.

Encoder parameters, optimizer moments, batch activations, and temporary matrix products are separate costs. After its first update, AdamW contributes about two parameter-sized moment buffers. The lateral state is not needed for inference, but it remains a training cost. Batch-size-one execution removes batch-dependent storage and temporal graphs; it does not remove the dense lateral matrix.

3 Relation to prior work

Two lines of work meet in TEREL. SFA supplies the structure of the local objective. Deep local representation learning supplies the credit-assignment setting. TEREL combines them through a particular soft objective and a specified execution boundary; it does not replace either foundation.

SFA and incremental SFA. Temporal slowness, variance preservation, and decorrelation define the SFA objective [Wiskott and Sejnowski \[2002\]](#). Linear batch SFA whitens the observations and extracts minor components of their temporal differences. IncSFA replaces these batch operations with incremental PCA and minor-component updates that never form a covariance matrix. IncSFA also handles episodic boundaries and has Hebbian and anti-Hebbian update forms [Kompella et al. \[2012\]](#). It is therefore the relevant online precedent rather than an alternative implementation of batch SFA. Our empirical baseline is a Python 3 port of the authors’ published linear algorithm. We document its provenance and validate it against batch SFA on a synthetic stream. TEREL instead applies a soft objective directly to each learned nonlinear layer and pays for a dense tracked lateral operator.

PowerSFA trains deep SFA models end to end by differentiating through an approximate whitening operation [Schüler et al. \[2018\]](#). Deep gradient-based SFA therefore predates TEREL. The distinction is that TEREL blocks gradients between learned layers and, in TEREL-S, across temporal history. Temporal coherence has also been used directly in deep encoders. Slow and Steady Feature Analysis, for example, regularizes first- and higher-order changes across video tuples [Jayaraman and Grauman \[2016\]](#). We restrict attention to first-order adjacency to study a neuron-local realization of soft constraints and bounded temporal state.

Bio-SFA and trace learning. Bio-SFA derives an online neural network for SFA with local synaptic updates [Lipshutz et al. \[2020\]](#). Thus neither online SFA nor local SFA is new here. TEREL differs in its deep soft-penalty parameterization and its explicit stop-gradient implementation for standard nonlinear layers. Földiák-style trace learning also exploits temporal persistence [Földiák \[1991\]](#). Its positive activation trace is algebraically distinct from the signed temporal difference in [Eq. \(8\)](#).

Gradient-isolated predictive and contrastive learning. Greedy InfoMax trains a stack of gradient-isolated modules with local InfoNCE objectives that predict along the natural order of images, audio, and video [Löwe et al. \[2019\]](#). It is the closest precedent for combining temporal structure with greedy depth-local representation learning. CLAPP makes contrastive prediction causal and local across both depth and time. Its update uses pre- and postsynaptic activity, predictive dendritic input, and broadcast modulation, but still requires positive and negative examples [Illing et al. \[2021\]](#). LoCo trains overlapping blocks with paired-view contrastive objectives. The loss of the next block can update parameters in the overlap, so LoCo relaxes strict gradient isolation [Xiong et al. \[2020\]](#).

Local temporal prediction and gradient-isolated deep learning are therefore prior art. TEREL studies a different objective: a noncontrastive soft-SFA loss with explicit variance and redundancy control. TEREL-S uses neither a negative set nor a predictive decoder, but it requires a dense, lagged,

Table 2: Comparison with the nearest objective and execution precedents. Auxiliary costs exclude learned feedforward parameters.

Method	Temporal objective	Execution	Learned mapping	Auxiliary state
Batch SFA	derivative minimization	batch eigensystem	linear	covariance and whitening
IncSFA	derivative minimization	sample-based	linear; deep cascades possible	incremental PCA and MCA state
Bio-SFA	derivative minimization	online neural dynamics	linear network in reported derivation	synaptic/dual variables
Greedy InfoMax	natural-order InfoNCE	greedy isolated modules	deep nonlinear modules	context/predictor and negatives
CLAPP	causal contrastive prediction	local in depth and time	deep nonlinear layers	predictive input, modulation, negatives
LoCo	paired-view contrastive	overlapping minibatch blocks	deep nonlinear blocks	projection heads and negatives
Blockwise SSL	paired-view invariance	mini-batch blocks	deep nonlinear blocks	batch statistics
TEREL family	temporal soft-SFA	sample or minibatch	deep nonlinear layers	$D^2 + 4D + 1$ per layer

same-layer operator. Its main MNIST experiment also uses labels to construct temporal order, unlike the principal unlabeled settings of these predictive methods.

Regularized and blockwise self-supervision. VICReg and Barlow Twins prevent collapse without negative examples by penalizing variance and covariance, or cross-correlation, respectively [Bardes et al. \[2022\]](#), [Zbontar et al. \[2021\]](#). Blockwise self-supervised learning has shown that VICReg, Barlow Twins, and contrastive losses can train network blocks without end-to-end backpropagation [Siddiqui et al. \[2024\]](#). TEREL differs in its temporal positive relation, detached running moments, and same-layer covariance-gradient proxy. These choices also permit one-observation-at-a-time execution.

The execution details further separate the methods. Siddiqui et al. find that simultaneous block training is materially stronger than sequential freezing, showing that forward co-adaptation remains useful after gradients are stopped at block boundaries [Siddiqui et al. \[2024\]](#). Canonical TEREL instead uses greedy layerwise training. Datta et al. apply VICReg losses to individual layers using original-augmented image pairs and current-batch variance and covariance terms [Datta et al. \[2025\]](#). TEREL uses a temporal relation, a gated detached variance state, and a lagged lateral gradient proxy.

The positive relation itself matters. Spectral analyses show that it can determine downstream geometry when it aligns with task labels [Balestriero and LeCun \[2022\]](#). Our class-chunk MNIST experiment must therefore be interpreted as constructed pair supervision and compared with a label-aware local baseline. It does not show that arbitrary natural adjacency reveals class structure.

4 Experimental protocol

The experiments examine four properties of the learning rule. The primary comparison measures canonical TEREL against backpropagation, two random encoders, and a label-aware local learner under the same constructed temporal supervision. A readout comparison tests whether the final layer retains the learned signal. A samplewise experiment tests whether TEREL-S can learn a non-collapsed representation with bounded state. Finally, a secondary wearable-sensor experiment asks whether the temporal mechanism transfers to a naturally ordered stream.

4.1 Data splits and roles

MNIST controlled temporal supervision. Using seed 1701, we stratify the official 60,000 MNIST training examples into 50,000 training and 10,000 validation examples. The official 10,000-example test split is reserved for final evaluation. At each encoder epoch, the training examples are randomly arranged into same-class chunks of length 16. Every chunk start defines a temporal boundary. Labels determine this order but never enter the TEREL objective. The protocol isolates the effect of a task-aligned temporal teaching signal and is not unlabeled self-supervision.

PAMAP2 secondary natural-order stress test. PAMAP2 provides 100-Hz wearable-sensor measurements and activity annotations for nine subjects [Reiss \[2012\]](#). We retain its 12 protocol activities, remove transition label 0 as directed by the dataset documentation, and downsample with stride 10. Missing sensor values are imputed with means from the training subjects, and all features are standardized with statistics from those subjects.

Subjects 1–6 train the encoder and probe, subject 7 provides validation data, and subject 8 is held out for testing. Subject 9 is excluded before model selection because its protocol recording contains only rope jumping and cannot support a 12-class test. Temporal boundaries occur at subject starts, gaps created by the removed transition rows, and timestamp discontinuities. The encoder iterator returns only features and boundary flags. Activity classes do not enter its loss, although transition annotation 0 affects preprocessing and boundary placement. This experiment tests the transfer of the temporal intuition; it does not replace the controlled MNIST comparison.

4.2 Models, baselines, and readouts

All primary encoders have dimensions $784 \rightarrow 512 \rightarrow 256$ and use LeakyReLU with negative slope 0.01. BatchNorm is a configuration choice rather than a component of the three-term objective. Canonical TEREL uses affine BatchNorm with update momentum 0.1, live temporal pairs within each chunk, and greedy training for 30 epochs per layer. The encoder consequently receives 60 complete presentations of the training set. AdamW uses learning rate 3×10^{-4} , weight decay 10^{-4} , and batch size 256. The loss coefficients are $(\lambda_S, \lambda_V, \lambda_C) = (1, 2.5, 1)$. The population momentum is 0.9. Setting $\rho_A = 0$ writes the lateral operator after each optimizer step, so the next minibatch uses the operator computed from the preceding minibatch.

The backpropagation baseline has the same hidden dimensions, activation, normalization, optimizer, batch size, and 60 data presentations. It is trained end to end with cross entropy. TEREL-S detaches the temporal references, trains both layers simultaneously with local objectives, omits BatchNorm, and uses coefficients $(1, 5, 1)$. Its separately validated schedule contains ten rather than 60 data presentations. We therefore report its resource use but do not describe it as data-presentation-matched to backpropagation.

Two random encoders distinguish learning from normalization. The first keeps its seed-specific hidden weights at initialization and omits normalization, matching the TEREL-S architecture. The second adds the affine BatchNorm modules used by canonical TEREL, but leaves its hidden weights, biases, BatchNorm scales, and BatchNorm offsets at initialization. A single `no_grad` pass over the 50,000 stored training examples, with batch size 256, updates only the running means and variances. The BatchNorm update momentum is 0.1, equivalent to retention 0.9. The encoder then enters evaluation mode, and the all-layer probe extracts representations using these fixed training-split statistics. The calibration is data-dependent normalization, not a parameter update.

Canonical minibatches concatenate randomly permuted, class-homogeneous chunks. A minibatch can therefore contain several classes, and BatchNorm communicates across all 256 examples in that minibatch. During greedy training, only the active layer’s BatchNorm remains in training

mode. The first layer’s running statistics are fixed before training the second layer. Representation extraction and probing use the final running statistics and learned affine parameters in evaluation mode. BatchNorm changes the within-layer information path, but it never transmits an error across depth.

Local SupCon provides a label-aware local comparison for the constructed MNIST protocol. Each layer minimizes a supervised contrastive loss over all same-label positives in its minibatch. Activations are detached between layers, and both layers update simultaneously. Local SupCon matches the primary architecture, BatchNorm, batch size, 60 data presentations, all-layer probe, and evaluation seeds. Validation searches a 3×2 grid with learning rates $\{10^{-4}, 3 \times 10^{-4}, 10^{-3}\}$ and temperatures $\{0.1, 0.2\}$. The resulting six candidates equal the method-selection budget used for TEREL.

Every primary method is evaluated by the same linear classifier on the concatenation of both hidden layers. A second probe reads only the final layer of the identical canonical TEREL encoder and tests depth specialization. All probes use AdamW for 60 epochs with learning rate 0.003, weight decay 10^{-4} , and batch size 2,048. Matching readout access prevents the breadth of the representation from being confounded with the training rule.

4.3 Model selection and evaluation separation

Model selection uses only the 10,000 validation examples and seeds 101, 202, and 303. After comparing candidate configurations on these seeds, we fix one configuration for each method before the five-seed evaluation. The Local SupCon learning-rate and temperature grid, the TEREL component interventions, and the lateral-signal controls remain validation-only. No reported test result selects among alternative configurations.

Batch-size-one execution changes both normalization and the number of optimizer updates, so the streaming configuration is selected separately. It uses TEREL-S, bounded running normalization, two data presentations, learning rate 6×10^{-6} , statistics and lateral momenta 0.99, and one forward and backward step per example. It accesses no future example and retains neither a temporal graph nor a buffer that grows with stream length.

4.4 Evaluation and uncertainty

The main evaluation uses seeds 1101, 1202, 1303, 1404, and 1505. For every method, we report the raw seed values, mean, sample standard deviation, a small-sample Student- t interval, and a percentile 95% bootstrap interval from 10,000 resamples with analysis seed 260803. Primary contrasts pair methods by seed. Student- t intervals are the principal intervals; bootstrap intervals provide a sensitivity analysis. Evidence for a learned representation is measured against both the normalization-matched and the no-normalization random encoders. A canonical seed is classified as non-collapsed only if its effective rank is at least 20, its median variance is at least 0.05, and its active fraction is at least 0.25.

The configurations, seeds, normalization calibration, and analysis rules are fixed before evaluation. Each selected method is evaluated once on the five test seeds. The BatchNorm-calibrated random control uses the same seeds without a validation sweep. Mechanism and direct-covariance analyses remain on the validation split and cannot select replacement models. The supplement contains the full selection ledger and all raw outputs.

5 Results

5.1 Competitive performance

Canonical TEREL attains $97.30 \pm 0.07\%$ test accuracy. Matched backpropagation attains $98.34 \pm 0.08\%$, giving a paired gap of 1.04 percentage points with 95% Student- t interval $[0.99, 1.08]$. Although the absolute accuracy gap is small, TEREL has 2.70% error compared with 1.66% for backpropagation. Its error rate is therefore 63% higher.

The BatchNorm-calibrated random encoder attains $95.35 \pm 0.19\%$. TEREL improves on this control by 1.95 points, with paired interval $[1.64, 2.25]$. The no-normalization random encoder attains $95.13 \pm 0.27\%$; calibration changes the random-encoder mean by 0.22 points, with interval $[-0.08, 0.52]$. Every TEREL seed satisfies the non-collapse criteria, and every intended layer and normalization parameter group receives a nonzero update. Thus a two-hidden-layer encoder trained by neuron-local soft-SFA reaches 97.30% in the constructed MNIST regime.

Table 3: MNIST test accuracy across five seeds. Every method uses the same concatenated-layer readout, except the explicitly marked final-layer TEREL row. Values report mean $\pm$ sample standard deviation.

Method	Accuracy
TEREL (all layers)	97.30 ± 0.07
TEREL (last layer)	97.14 ± 0.10
TEREL-S (all layers)	96.29 ± 0.07
Random, no normalization (all layers)	95.13 ± 0.27
Backpropagation (all layers)	98.34 ± 0.08

Table 4: Paired differences in MNIST test accuracy. Values are percentage points with 95% Student- t intervals over five seed-paired differences.

Contrast	Difference [95% interval]
TEREL – BP	$-1.04 [-1.08, -0.99]$
TEREL – random, no normalization	$2.17 [1.83, 2.51]$
last – all layers	$-0.16 [-0.37, 0.05]$

Table 5: MNIST accuracy of the normalization-matched random control across five paired seeds. Hidden weights and affine BatchNorm parameters remain at initialization; only running statistics are calibrated on the training split. Method rows report mean $\pm$ sample standard deviation, and contrasts report 95% Student- t intervals.

Method or contrast	Accuracy / difference
Random, no normalization	95.13 ± 0.27
BatchNorm-calibrated random	95.35 ± 0.19
TEREL	97.30 ± 0.07
TEREL – calibrated random	$1.95 [1.64, 2.25]$
calibrated – no-normalization random	$0.22 [-0.08, 0.52]$

5.2 Matched label-aware local comparator

The validation-selected Local SupCon baseline attains $96.98 \pm 0.10\%$ on the same five test seeds. Canonical TEREL is higher by 0.32 points, with paired Student- t interval $[0.13, 0.52]$. Local SupCon uses every same-label positive in its minibatch, whereas TEREL observes only adjacent examples within the label-constructed chunks. A generic layer-local same-class objective therefore does not reproduce the TEREL result under this protocol. The 0.32-point difference is specific to this benchmark and does not establish general superiority over local contrastive learning.

Table 6: MNIST test accuracy for the matched label-aware comparison across five seeds. Local SupCon uses all same-label positives in each minibatch. Both methods stop credit at layer boundaries and use the same architecture, 60 data presentations, and concatenated-layer probe.

Method	Accuracy
TEREL	97.30 ± 0.07
Local SupCon	96.98 ± 0.10
TEREL – Local SupCon	0.32 [0.13, 0.52]

The bracketed value is a paired 95% Student- t interval.

5.3 Mechanism isolation

The validation-only mechanism study changes one factor at a time under the canonical protocol. Removing the temporal term reduces accuracy by 8.18 points without collapsing effective rank. Shuffling the class-persistent order reduces accuracy by 7.55 points. Temporal coherence therefore supplies the task-aligned teaching signal in this controlled experiment.

The other two terms prevent different degeneracies. Without gated variance expansion, the median feature variance falls to 7.7×10^{-5} . Without decorrelation, feature variance remains high but effective rank falls to 2.8. BatchNorm alone prevents neither failure. These interventions assign distinct empirical functions to the temporal, variance, and decorrelation terms, as predicted by the soft-SFA formulation.

Table 7: One-factor mechanism interventions on the validation split across three paired seeds. Δ is the accuracy difference from full TEREL in percentage points.

Intervention	Accuracy (%)	Δ	r_{eff}	Median var.
Full TEREL	97.28 ± 0.12	–	39.2	0.185
$\lambda_S = 0$	89.10 ± 1.57	-8.18	369.3	0.907
$\lambda_V = 0$	66.13 ± 6.32	-31.15	6.0	7.72e-05
$\lambda_C = 0$	86.44 ± 0.41	-10.84	2.8	0.916
Shuffled order	89.73 ± 0.37	-7.55	311.3	0.171

The lateral-operator analysis measures 17,637 nonzero minibatch directions per layer over three validation seeds. In both layers, the preceding-minibatch direction has mean cosine alignment 0.76 with the same-minibatch direction. Its relative ℓ_2 error is 0.65, and its norm is approximately 0.78 times the norm of the target direction. The lagged operator is therefore directionally informative but not equivalent to the same-batch covariance gradient.

We also replace the proxy by the exact differentiable same-batch penalty and match the factor-four gradient scale. This direct control attains $96.93 \pm 0.07\%$ validation accuracy, compared with $97.19 \pm 0.09\%$ for the measured lagged configuration. The paired direct-minus-lagged difference is -0.25

points, with 95% Student- t interval $[-0.42, -0.08]$. The measured discrepancy rules out identifying the lagged state with the exact gradient, while the accuracy comparison shows that the result does not require the exact same-batch operator.

Table 8: Alignment of the lagged lateral direction with its same-batch target on the validation split. Statistics average three seeds and all nonzero minibatches included in the analysis.

Layer	Cosine alignment	Relative ℓ_2 error	Norm ratio
1	0.757	0.651	0.785
2	0.755	0.653	0.780

5.4 Depth, readout, and representation geometry

The final-layer probe attains $97.14 \pm 0.10\%$, 0.16 points below the matched all-layer probe. The paired interval $[-0.37, 0.05]$ includes zero. The final layer therefore retains nearly all of the mean result, although five seeds do not resolve the small difference between readouts. In particular, the primary accuracy is not obtained solely by concatenating a strong first layer with a weak second layer.

After ten joint-training presentations, TEREL-S attains $96.29 \pm 0.07\%$. It exceeds its unnormalized random reference by 1.16 points, with paired interval $[0.94, 1.45]$. Despite its stricter temporal boundary and shorter schedule, TEREL-S learns a useful representation.

The geometry diagnostics support the probe results. Canonical TEREL increases nearest-centroid accuracy from 80.21% for the no-normalization random encoder to 95.55%, and increases the between-class to within-class scatter ratio from 0.24 to 0.70. Its final layer is more concentrated by class, with 96.22% centroid accuracy and scatter ratio 1.52. The concatenated representation retains greater effective rank. This pattern is consistent with a soft-SFA tradeoff: the representation exposes persistent factors without compressing every output direction into the same class geometry.

Table 9: Geometry of the MNIST representations, averaged across five seeds. B/W denotes the between-class to within-class scatter ratio; Sel. denotes median unit class selectivity; r_{eff} is covariance effective rank; and $|\rho_{\text{off}}|$ is mean absolute off-diagonal correlation.

Method	Centroid (%)	B/W	Sel.	r_{eff}	$ \rho_{\text{off}} $
Random, no norm. (all)	80.21	0.241	1.239	33.84	0.101
TEREL-S (all)	92.04	0.101	0.843	142.32	0.057
TEREL (all)	95.55	0.695	1.704	37.35	0.103
TEREL (last)	96.22	1.516	2.567	18.86	0.161
BP (all)	97.87	0.968	1.857	21.17	0.152

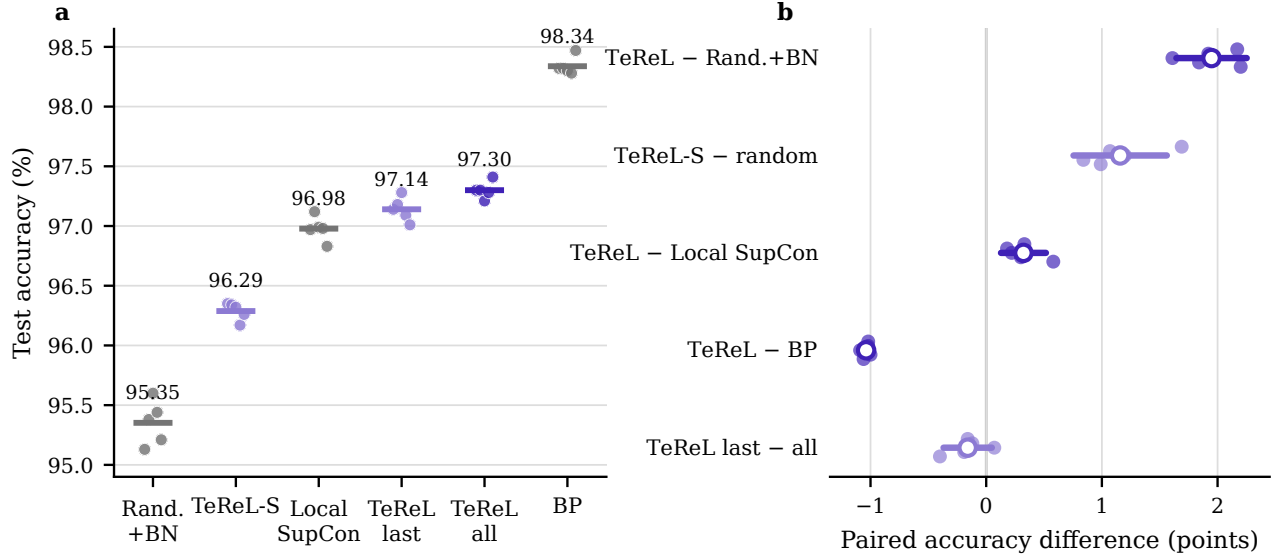


Figure 2: MNIST test accuracy over five paired seeds. **a:** Individual seed values and means (horizontal bars). Canonical TeReL lies below backpropagation and above the normalization-matched random encoder and Local SupCon. Its final-layer probe retains the mean accuracy, and TeReL-S exceeds the no-normalization random encoder. **b:** Paired differences in the subtraction order printed on each row. Open circles mark means; lines show 95% Student- t intervals.

5.5 Natural temporal order on PAMAP2

The PAMAP2 comparison changes only the order presented to the encoder. Architecture, initialization, optimizer, sample count, and probe remain fixed, and activity classes are absent from encoder batches. Chronological order attains macro-F1 0.307; shuffled order attains 0.312. Their paired interval, $[-0.063, 0.053]$, includes advantages in either direction. Every method also shows a large shift from validation subject 7 to test subject 8. This single heterogeneous split therefore resolves neither an ordered nor a shuffled advantage. Testing natural-order transfer requires several validation subjects and tighter control of subject shift.

5.6 Streaming execution and cost

Validation selection for the samplewise configuration varies the learning rate, state timescale, streaming normalizer, gradient accumulation, and number of data presentations. The selected two-pass configuration attains $95.14 \pm 0.04\%$ validation accuracy over three seeds. Its effective rank is 112.8, with seed range 106.6–120.9, and its median feature variance is 0.378. It exceeds the matched no-normalization random validation mean by 0.29 points without retaining a temporal graph.

At each step, the preceding activation, running moments, normalization state, and lateral matrix are detached. The state size is independent of stream length. The resource-measurement seed performs one optimizer update per observation and requires 1,305 seconds. This time reflects substantial launch and update overhead in the stock GPU/PyTorch implementation. The experiment establishes that samplewise learning can preserve a useful representation with bounded storage; it establishes neither throughput nor energy efficiency. The resource measurements for the main methods appear in [Tables 12 and 14](#).

6 Limitations

The method assumes that temporal neighbors share a latent cause. Rapidly changing or aliased causes can violate this assumption and make adjacency misleading. The MNIST ordering is synthetic and label-derived, so its accuracy measures learning from a constructed temporal signal rather than label-free learning. PAMAP2 is a small sensor benchmark with substantial subject shift, and one subject-disjoint split cannot resolve the broader natural-order question. Images, video, audio, and other natural streams must be evaluated directly. Any deployment on human activity data would also require consent, privacy protection, and analysis of surveillance risk.

Bounded temporal history does not make the method cheap in layer width. The dense lateral operator requires $O(D^2)$ state and within-layer communication. Sparse and low-rank alternatives require direct tests of representation quality. The samplewise result is a validation experiment on MNIST, not a five-seed test evaluation or a throughput-matched comparison. Its one-sample optimizer updates are slow in the present GPU implementation. Canonical TEREL likewise takes more wall-clock time than backpropagation despite its lower declared operation proxy. The software measurements support no claim of hardware or energy efficiency.

BatchNorm in canonical TEREL communicates across examples at each neuron, and its affine parameters are learned. Neuron-locality is therefore a spatial claim, not a claim that the primary update is per-example. The simpler outer-product factorization applies only to variants with the identity normalizer. More generally, the locality claim concerns the path of credit assignment. The implementation still relies on standard optimizers, exact activation derivatives, normalization, and synchronized matrix operations.

7 Perspective and future directions

Natural order. SFA is motivated by streams in which stable causes generate changing observations, not by class-sorted images. PAMAP2 provides no evidence that adjacency helps under severe subject shift. More informative settings include video with controlled viewpoint change, egocentric interaction, and continuous audio. In these settings, the timescale of the latent cause can be compared directly with the state and chunk timescales of TEREL. The next experiment should vary these timescales rather than summarize natural order by a single chronological-versus-shuffled contrast.

Constrained hardware. With identity normalization, each update requires the activity before the synapse, a postsynaptic teaching factor, leaky state, and no error path across depth. This dependency structure is compatible with neuron-local or event-driven execution. TEREL-S also removes the temporal graph, and the samplewise experiment shows that this stricter form can retain useful accuracy. Compatibility does not imply energy efficiency. A hardware study would need low-rank lateral state, fused state updates, and an optimizer designed for local arrays.

Recurrent extension. The temporal objective suggests a recurrent module whose slow representation supports prediction without receiving gradients from the prediction path. A valid test must isolate the representation and prediction pathways, include matched random-state and backpropagation-through-time controls, and repeat the comparison across seeds. This recurrent construction is a hypothesis implied by the mechanism, not a result of the present study.

8 Conclusion

This work asks whether the SFA principle can train a deep nonlinear encoder without propagating errors across layers. TEREL answers by assigning each layer a temporal-coherence term, a gated variance-expansion term, and a same-layer decorrelation signal. Canonical TEREL preserves short temporal gradients within a chunk while blocking gradients across depth. TEREL-S also detaches the temporal reference and retains state that is independent of stream length, although its dense lateral matrix remains quadratic in width.

Under the controlled MNIST ordering, canonical TEREL attains 97.30%, 1.04 points below data-presentation-matched backpropagation and 1.95 points above the BatchNorm-calibrated random encoder. The final layer alone attains 97.14%. TEREL-S attains 96.29%, and its samplewise configuration attains 95.14% validation accuracy with effective rank 112.8. Local SupCon attains 96.98%, 0.32 points below TEREL over the paired seeds. The mechanism controls separate the roles of the three signals: temporal coherence carries the task-aligned relation, variance expansion prevents scale collapse, and decorrelation prevents rank collapse.

These results show that temporal coherence, when protected against collapse and redundancy, can train a two-hidden-layer representation with neuron-local credit assignment. This claim is confined to a constructed, label-derived temporal relation. The PAMAP2 comparison does not show that the same advantage transfers to natural streams. That question requires broader experiments that control distribution shift and relate the timescale of the data to the timescale of the learning rule.

References

- Laurenz Wiskott and Terrence J Sejnowski. Slow feature analysis: Unsupervised learning of invariances. *Neural computation*, 14(4):715–770, 2002.
- Adrien Bardes, Jean Ponce, and Yann LeCun. VICReg: Variance-Invariance-Covariance Regularization for Self-Supervised Learning. *arXiv preprint arXiv:2105.04906*, 2022.
- Jure Zbontar, Li Jing, Ishan Misra, Yann LeCun, and Stéphane Deny. Barlow Twins: Self-Supervised Learning via Redundancy Reduction. *arXiv preprint arXiv:2103.03230*, 2021.
- Varun Raj Kompella, Matthew Luciw, and Jürgen Schmidhuber. Incremental Slow Feature Analysis: Adaptive and Episodic Learning from High-Dimensional Input Streams. *Neural Computation*, 24(11):2994–3024, 2012. doi: 10.1162/NECO_a_00344.
- Merlin Schöler, Hlynur Davíð Hlynsson, and Laurenz Wiskott. Gradient-based Training of Slow Feature Analysis by Differentiable Approximate Whitening. *OpenReview preprint*, 2018. URL <https://openreview.net/forum?id=rJlHlo09KQ>.
- Dinesh Jayaraman and Kristen Grauman. Slow and Steady Feature Analysis: Higher Order Temporal Coherence in Video. In *IEEE Conference on Computer Vision and Pattern Recognition*, pages 3852–3861, 2016.
- David Lipshutz, Charles Windolf, Siavash Golkar, and Dmitri Chklovskii. A Biologically Plausible Neural Network for Slow Feature Analysis. In *Advances in Neural Information Processing Systems*, volume 33, pages 14986–14996, 2020.
- P Földiák. Learning invariance from transformation sequences. *Neural computation*, 3(2):194–200, 1991.

- Sindy Löwe, Peter O'Connor, and Bastiaan S. Veeling. Putting An End to End-to-End: Gradient-Isolated Learning of Representations. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- Bernd Illing, Jean Ventura, Guillaume Bellec, and Wulfram Gerstner. Local plasticity rules can learn deep representations using self-supervised contrastive predictions. In *Advances in Neural Information Processing Systems*, volume 34, 2021.
- Yuwen Xiong, Mengye Ren, and Raquel Urtasun. LoCo: Local Contrastive Representation Learning. In *Advances in Neural Information Processing Systems*, volume 33, 2020.
- Shoaib Ahmed Siddiqui, David Krueger, Yann LeCun, and Stéphane Deny. Blockwise Self-Supervised Learning at Scale. *Transactions on Machine Learning Research*, 2024.
- Joy Datta, Rawhatur Rabbi, Puja Saha, Aniqua Nusrat Zereen, M. Abdullah Al-Wadud, and Jia Uddin. Deep Representation Learning Using Layer-Wise VICReg Losses. *Scientific Reports*, 15:27049, 2025. doi: 10.1038/s41598-025-08504-2.
- Randall Balestriero and Yann LeCun. Contrastive and Non-Contrastive Self-Supervised Learning Recover Global and Local Spectral Embedding Methods. In *Advances in Neural Information Processing Systems*, volume 35, pages 26671–26685, 2022. URL <https://openreview.net/forum?id=jQgsZDspz5h>.
- Attila Reiss. PAMAP2 Physical Activity Monitoring, 2012.

A Gradient derivations

For a valid pair with a detached temporal reference,

$$\frac{\partial}{\partial z_t} \frac{1}{N_I D} \sum_s I_s \|z_s - \text{sg}(z_{s-1})\|^2 = \frac{2I_t}{N_I D} (z_t - p_t). \quad (14)$$

Because g , m , A , and q_t are detached,

$$\frac{\partial L_V}{\partial z_t} = -\frac{2}{BD} g \odot (z_t - m), \quad (15)$$

$$\frac{\partial L_C}{\partial z_t} = \frac{1}{BD} A q_t. \quad (16)$$

Adding these derivatives yields Eq. (8). The chain rule through ϕ and then through the minibatch map $N_{\eta, B}$ yields the Jacobian-block expression in Eq. (9). Setting N to the identity recovers the presynaptic outer-product special case. With an undetached temporal reference, z_t appears as the current element of pair t and as the preceding element of pair $t + 1$. The sum of these two derivatives gives Eq. (10). Finite-fixture regression tests verify the two signs and the future-activation term.

For the direct covariance control, let Z_c stack centered rows and $C = \text{offdiag}(Z_c^\top Z_c / B)$. Since C is symmetric,

$$\frac{\partial}{\partial Z_c} \frac{\|C\|_F^2}{D} = \frac{4}{BD} Z_c C, \quad (17)$$

which is $4C(z_t - m)/(BD)$ in column-vector notation. TEREL replaces $4C(z_t - m)$ by the detached direction $A q_t$ and absorbs the positive factor into λ_C .

B Implementation-fidelity tests

The public test suite verifies the following properties:

- the sign and gate of the variance gradient, detachment of the temporal reference, the future term for undetached time, the lateral-proxy error bound, and the direct-covariance derivative;
- nonzero updates to every declared encoder layer and lateral state, and the absence of a gradient from a later layer to an earlier layer;
- fixed dynamic-state size, bounded streaming normalization, release of samplewise gradients, and the absence of retained autograd graphs;
- training-split-only BatchNorm calibration that changes running buffers while preserving every random-encoder weight, bias, and affine parameter;
- the MNIST split roles, PAMAP2 subject roles, normalization fit only on the training split, temporal boundaries, and encoder batches containing only features and boundary batches;
- matched all-layer and final-layer probes, greedy layerwise scheduling, execution of every declared backpropagation layer, and exact counting of data presentations;
- batch-SFA constraints, boundary-aware derivatives, and numerical agreement of the published-algorithm IncSFA port with its reference; and
- rejection of held-out evaluation after any change to the protocol, selection ledger, configuration, or tracked source.

C Reproducibility record

The repository fixes Python 3.12 and all direct dependencies through `pyproject.toml` and `uv.lock`. Every run enables deterministic PyTorch and cuDNN behavior together with the prescribed cuBLAS workspace configuration. The anonymous supplement contains the selected configurations, selection ledger, seeds, per-seed JSON records, raw confusion matrices, hardware and software record, source, generated tables, and checksums. `ARTIFACT_README.md` provides portable commands and the exact provenance identifiers. The supplement does not redistribute dataset files.

D Secondary natural-stream controls

The PAMAP2 encoder never receives activity-class labels. The chronological and shuffled TEREL conditions differ only in encoder order. The other rows compare them with untrained features, classical SFA, the exact in-batch covariance control, and end-to-end supervision. Every method performs weakly on subject 8, consistent with substantial cross-subject shift.

Table 10: PAMAP2 test macro-F1 across five seeds in the secondary natural-stream experiment. Values report mean $\pm$ sample standard deviation and percentile 95% bootstrap interval.

Method	Macro-F1	
Chronological TEREL	0.307 ± 0.066	[0.260, 0.360]
Shuffled TEREL	0.312 ± 0.026	[0.290, 0.330]
Random features	0.322 ± 0.039	[0.296, 0.355]
Batch SFA	0.185 ± 0.002	[0.184, 0.186]
Incremental SFA	0.218 ± 0.015	[0.206, 0.229]
Direct covariance	0.243 ± 0.021	[0.226, 0.259]
Supervised BP	0.319 ± 0.030	[0.296, 0.341]

E Complete configurations and raw results

Table 11: MNIST test accuracy for seeds 1101, 1202, 1303, 1404, and 1505, in that order.

Method	Raw values
TEREL (all layers)	97.30, 97.30, 97.21, 97.28, 97.41
TEREL (last layer)	97.14, 97.18, 97.28, 97.09, 97.01
TEREL-S (all layers)	96.35, 96.34, 96.32, 96.17, 96.26
Random, no normalization (all layers)	94.66, 95.27, 95.12, 95.33, 95.27
Backpropagation (all layers)	98.32, 98.32, 98.30, 98.28, 98.47

Table 12: Resource use during encoder fitting, averaged across five seeds. Examples count sample presentations. MAC denotes the declared linear and same-layer operation proxy, not energy use.

Method	Examples (M)	Opt. steps	Seconds	Peak MiB	MAC (G)
TEREL (all layers)	3.00	11760	107.6	76.7	3981.3
TEREL (last layer)	3.00	11760	107.7	76.7	3981.3
TEREL-S (all layers)	0.50	1960	27.4	77.7	1126.4
Random, no normalization (all layers)	0.00	0	0.0	0.0	0.0
Backpropagation (all layers)	3.00	11760	43.4	225.4	4815.4

Table 13: Training state in MiB. Model parameters include every parameter fitted during encoder training, including the supervised head for backpropagation. Algorithm state contains the TEREL lateral and statistical state but excludes normalization buffers. Optimizer and algorithm state are training-only. The inference encoder retains hidden-layer parameters and normalization buffers, but discards the supervised head and TEREL state.

Method	Model params	Optimizer	Algorithm	Norm.	Inference encoder
TEREL (all layers)	2.040	4.080	1.262	0.006	2.046
TEREL (last layer)	2.040	4.080	1.262	0.006	2.046
TEREL-S (all layers)	2.034	4.068	1.262	0.000	2.034
Random, no normalization (all layers)	2.034	0.000	0.000	0.000	2.034
Backpropagation (all layers)	2.050	4.100	0.000	0.006	2.046

Table 14: Samplewise TEREL-S on the validation split across three seeds. The serial run provides the time and memory measurements; timings from concurrent repetitions are excluded.

Accuracy (%)	Effective rank	Median variance	Encoder seconds
95.14 ± 0.04	112.8	0.378	1305.2

Table 15: Local SupCon validation grid across three seeds. Bold marks the configuration selected before test evaluation.

Learning rate	Temperature	Accuracy
1e-04	0.1	96.49 ± 0.15
1e-04	0.2	95.60 ± 0.12
3e-04	0.1	96.63 ± 0.03
3e-04	0.2	95.98 ± 0.11
1e-03	0.1	96.90 ± 0.03
1e-03	0.2	96.20 ± 0.02

Table 16: Local SupCon test values and resource state across five seeds. Inference retains the encoder parameters and BatchNorm buffers; optimizer state is used only during training.

Quantity	Value
Test accuracy by seed (%)	96.97, 97.12, 96.99, 96.98, 96.83
Mean encoder seconds	82.1
Model parameters (MiB)	2.040
Optimizer state (MiB)	4.080
Normalization buffers (MiB)	0.006
Inference encoder (MiB)	2.046